

Reviewer Pack — Minimal Rank of Geometric Langlands Duality over Tseitin Res...

Entry 188d910e7e09 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 03:56:15 UTC

Minimal Rank of Geometric Langlands Duality over Tseitin Resolution Trees Complexity

Entry ID: 188d910e7e09 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 03:56:15 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Program **Field B** (complexity object): Complexity Theory: Tseitin Resolution Trees

Statement:

[‘For every Tseitin resolution tree T rooted at a 3-CNF formula, the minimal rank of its associated geometric Langlands dual object is upper bounded by the depth of T .’, ‘Equivalently, there exists an absolute constant c such that for all Tseitin resolution trees T with depth d , the rank of the geometric Langlands dual object of T is at most cd .’]

Rationale (proposer’s reasoning):

[‘The Geometric Langlands Program provides a bridge between algebraic geometry and representation theory, which may offer new insights into complexity theory. By associating geometric Langlands dual objects with Tseitin resolution trees, we could potentially uncover hidden structures that are not apparent in the original tree structure.’, ‘If this conjecture holds, it would suggest that the geometric Langlands program can be used to derive lower bounds on the complexity of certain computational problems.’]

Taxonomy category: Geometric_Langlands_Program (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2f1f0f28e70d4235

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all generated Tseitin resolution trees, the ratio of the rank of the geometric Langlands dual object to the tree depth is less than or equal to a constant c , and falsified if any tree has a ratio exceeding c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - geometric langlands program AND tseitin resolution trees complexity
- tseitin resolution tree depth AND geometric langlands dual object rank - upper bound ON
geometric langlands dual rank BY tseitin resolution tree depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_resolution_tree(clauses):
        if not clauses:
            return {}, {}

        literals = set()
        tree = {lit: [] for lit in literals}

        def add_clause(clause):
            nonlocal literals
            new_var = len(literals) + 1
            literals.add(new_var)

            for lit in clause:
                if lit < 0:
                    tree[-lit].append((new_var, 'not'))
                else:
                    tree[lit].append((new_var, 'and'))

            add_clause([-new_var])

        for clause in clauses:
            add_clause(clause)
```

```

    return tree, literals

def geometric_langlands_rank(tree):
    # Placeholder implementation
    # This should be replaced with a proper mapping to an algebraic structure
    # and computation of the rank of the associated object.
    return len(tree)

n = random.randint(5, 40)
clauses = []
for _ in range(n):
    num_vars = random.randint(2, 10)
    clause = [random.choice([-i, i]) for i in range(1, num_vars + 1)]
    clauses.append(clause)

tree, literals = tseitin_resolution_tree(clauses)
rank = geometric_langlands_rank(tree)
depth = max(len(path) for path in tree.values() if path)

metric_value = rank / depth
conjecture_holds = all(metric_value <= c for c in [1.0, 2.0, 3.0]) # Example constants
counterexample = "" if conjecture_holds else f"Rank {rank} exceeds depth {depth}"

return {
    "metric_name": "Ratio of Rank to Depth",
    "metric_value": metric_value,
    "instances_tested": len(clauses),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6972f09e.py", line 81, in <module>
    result = run_trial(seed)
            ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6972f09e.py", line 59, in run_trial
    tree, literals = tseitin_resolution_tree(clauses)
                    ~~~~~^~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6972f09e.py", line 42, in tseitin_resolution
    add_clause(clause)
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6972f09e.py", line 37, in add_clause
    tree[lit].append((new_var, 'and'))
    ~~~~^~~~~~
```

KeyError: 1

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify the conjecture's support or falsification. | next: Re-run the test to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	?	?	0	0	15202
2	propose	ollama_remote	glm4:latest	0	0	28047
3	preregistration	ollama_remote	glm4:latest	0	0	5575
4	novelty	ollama_remote	glm4:latest	0	0	4662
5	novelty	ollama_remote	glm4:latest	0	0	10309
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27307
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10600
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10177
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8726
10	judge	ollama_remote	glm4:latest	0	0	8214

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128819 ms total latency. Provider mix: {'?': 1, 'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/188d910e7e09.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/188d910e7e09.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/188d910e7e09.tar.gz` (if generated)